



UNIVERSITY OF TORONTO

AER1515 Perception for Robotics: Assignment 2

Student : YANG-CHEN LIN

November 1, 2025

Contents

1	Answers	2
1.1	Question 1: Feature Point Detection and Correspondences	2
1.1.1	Feature Detection	2
1.1.2	Feature Matching	7
1.1.3	Outlier Rejection	10
1.1.4	Alternative Algorithm	12
1.2	Question 2: 3D Point Cloud Registration	17
1.2.1	Nearest Neighbour Search	17
1.2.2	Pose Estimation from the Correspondences	19
1.2.3	Iterative Closest Point Registration	21

1 Answers

1.1 Question 1: Feature Point Detection and Correspondences

1.1.1 Feature Detection

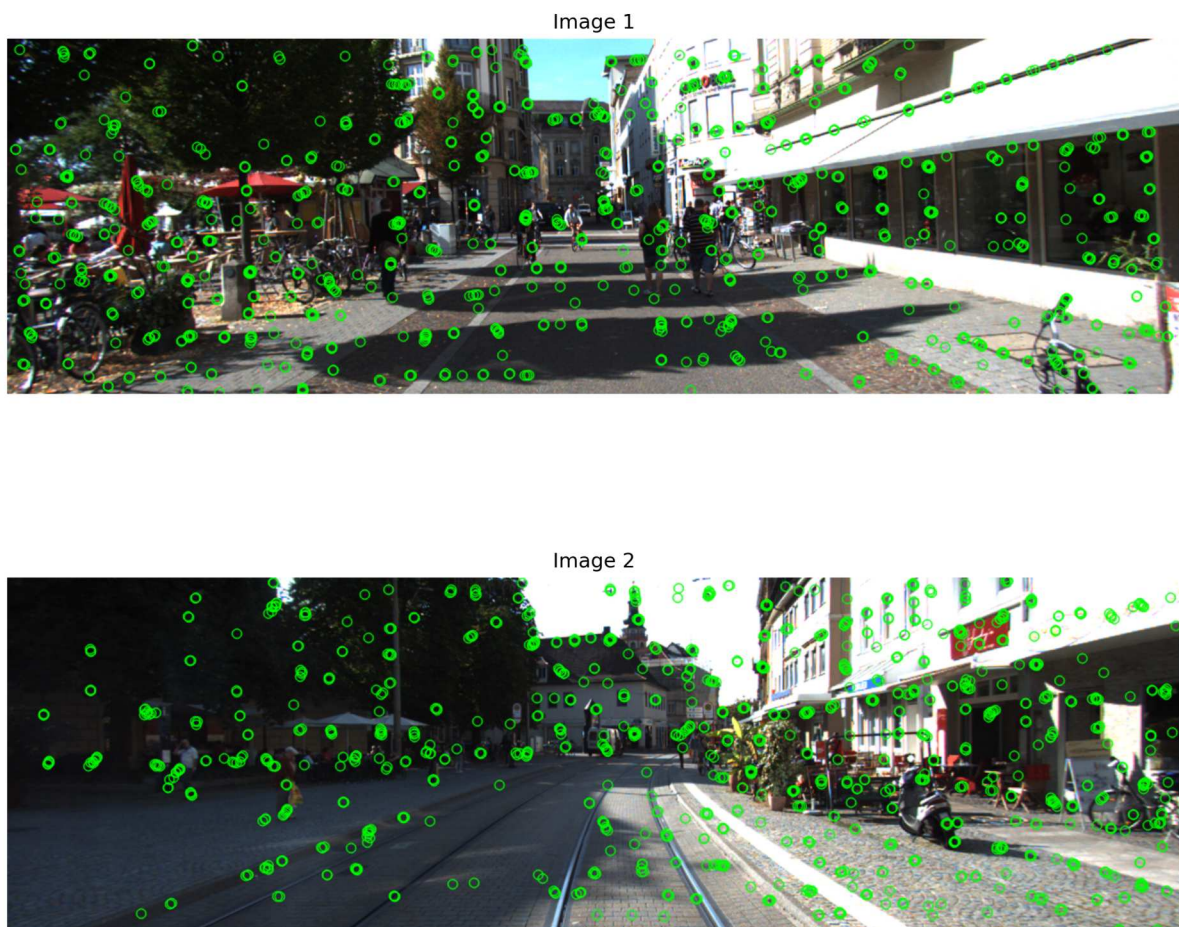


Figure 1: Detected keypoints of left-test-images by the R2D2 framework (unified circle radius)

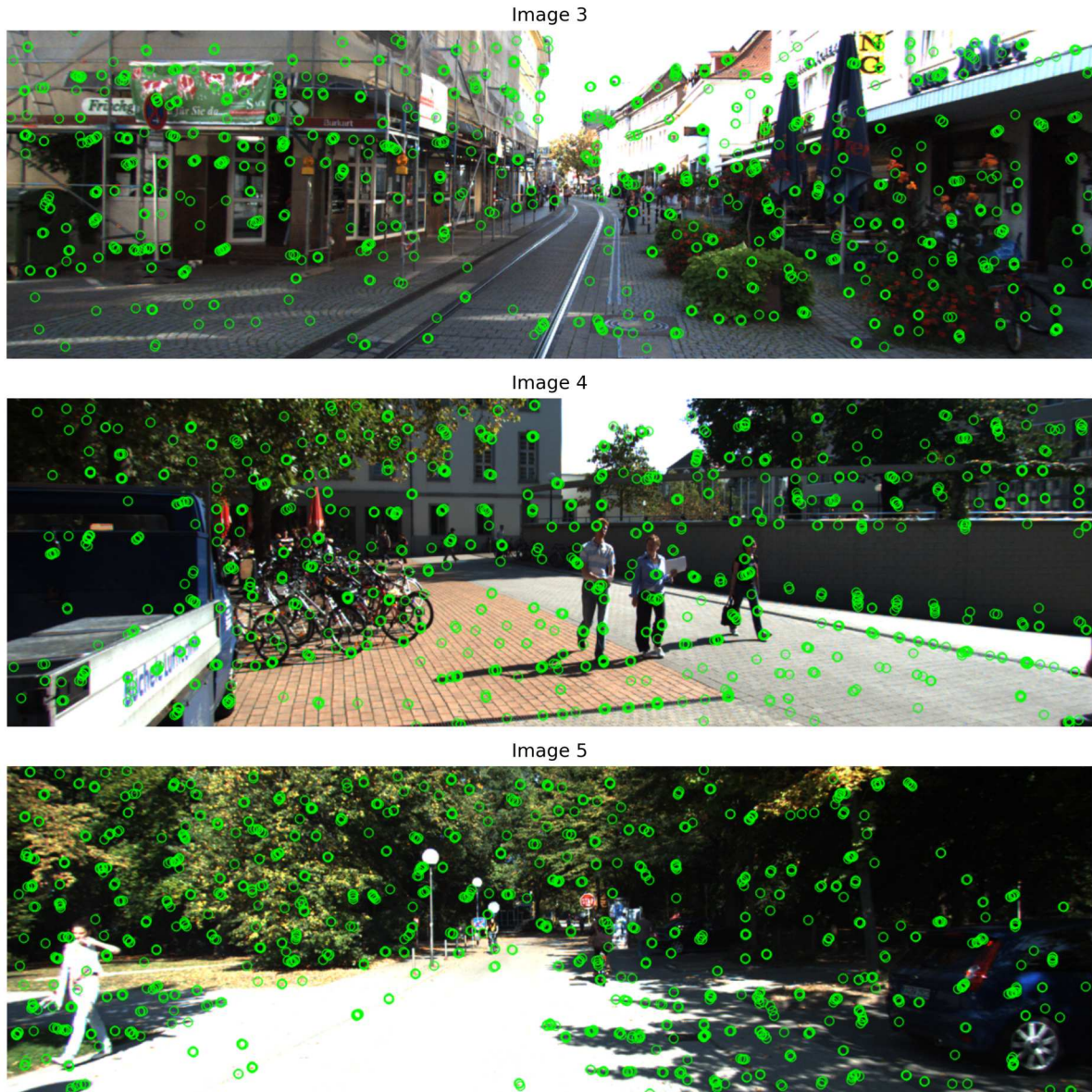


Figure 2: Detected keypoints of left-test-images by the R2D2 framework (unified circle radius)

R2D2: Feature Detector Summary For the first feature detector, I chose the R2D2 detector from [1]. The R2D2 detector-descriptor is a CNN-based method that jointly predicts (1) a repeatability heatmap $R(\mathbf{x})$, (2) a reliability/confidence map $S(\mathbf{x})$, and (3) dense descriptors $D(\mathbf{x})$ over the image. Keypoints are produced by selecting spatial local maxima of the repeatability map and then filtering those candidates by reliability:

$$\mathcal{K} = \{ \mathbf{x} \mid R(\mathbf{x}) \text{ is a local maximum and } S(\mathbf{x}) > \tau \},$$

where τ is a threshold controlling how reliable a detected point must be to be kept. Descriptors are sampled from the dense descriptor field and L2-normalized:

$$\mathbf{d}(\mathbf{x}) = \frac{D(\mathbf{x})}{\|D(\mathbf{x})\|_2}.$$

During training, R2D2 optimizes three complementary objectives: a descriptor matching loss that encourages descriptors at corresponding locations to be similar, a repeatability loss that encourages stable local maxima under geometric/image transformations, and a reliability loss that predicts whether a detected keypoint will yield a correct match. In practice, this yields sparse keypoints that are both repeatable across views and predictable to match (high reliability), which improves downstream matching quality compared with detectors that report only corner strength or only descriptor distinctiveness.

R2D2: Unreliable Keypoints From Fig. 1 and Fig. 2, we can observe that most keypoints are concentrated around visually distinctive regions of the image, such as object boundaries, textured surfaces, and high-contrast areas. The clustering of keypoints occurs because R2D2 detects features across multiple scales, which can lead to overlapping detections when strong structures are present at several pyramid levels. However, there are still few keypoints that lie in regions that have less distinct or repetitive features, such as the ones in the shaded tree area on the left side of Image 2 and right side of Image 5, or on the ground piles in most of the images. In the following plots, the radius of each circle represents the detected scale of the corresponding keypoint, illustrating the multi-scale nature of the detector.



Figure 3: Top 30 keypoints with scaled radius in each image



Figure 4: Top 30 keypoints with scaled radius in each image

The radius of each green circle represents the scale predicted by R2D2 for that keypoint, corresponding to the size of the image region that contributes to its descriptor. Larger circles indicate coarse-scale features that remain stable across viewpoint and blur changes, while smaller circles correspond to fine-scale texture details. Since the R2D2 framework natively provide "scores" for each keypoint, it's easy to identify which keypoints are less reliable when we perform image matching in the later steps.

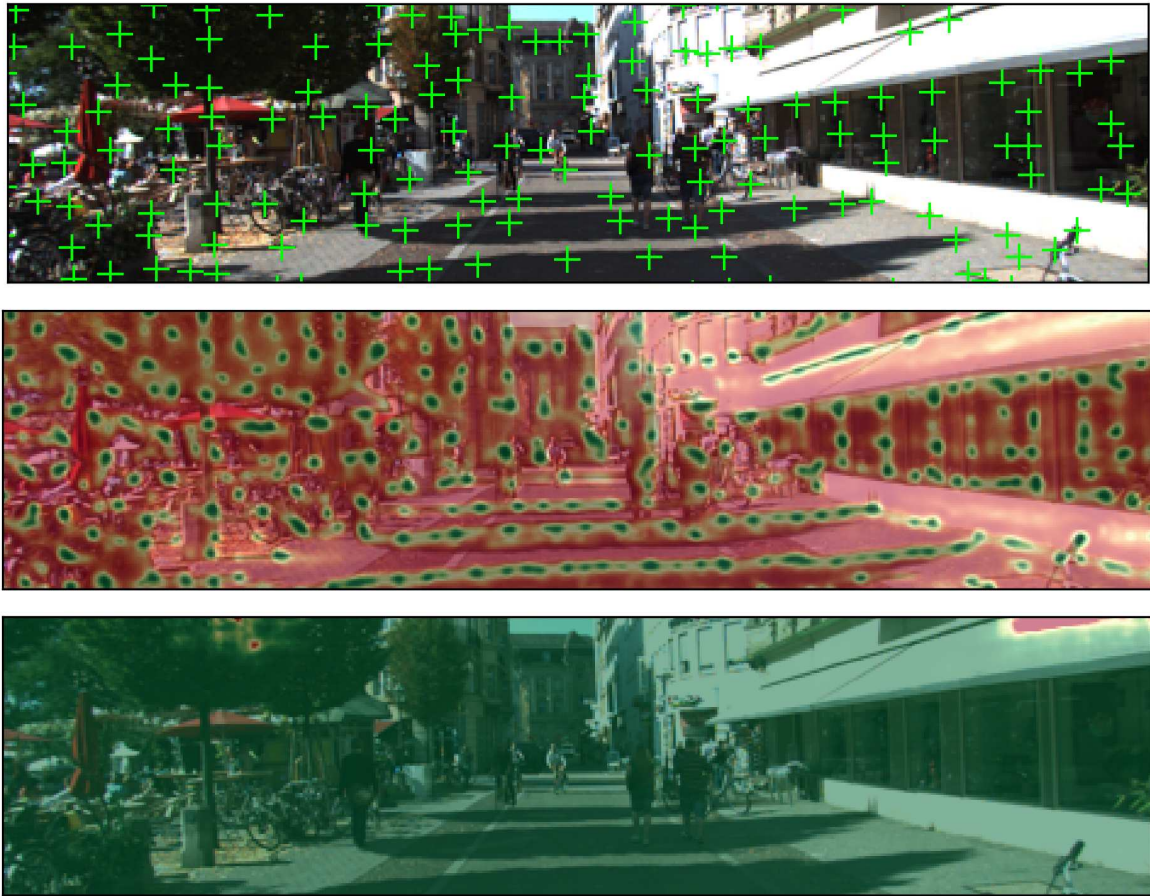


Figure 5: For one given input image (1st row), the repeatability (2nd row) and reliability heatmaps (3rd row) extracted at a single scale. Top-scoring keypoints are shown as green crosses in the first image. (Plot generated by visualization script from the [R2D2 repository](#).)

1.1.2 Feature Matching

The feature matching pipeline used in this part is implemented across three scripts in the repository: `tools/R2D2.py`, `tools/matcher.py`, and `starter_code_feature.py`. The following summarizes the implementation, parameters, and the filtering rules applied during matching and depth computation.

Overview

- `tools/R2D2.py`: performs feature detection and descriptor extraction using the R2D2 framework [1]. It produces per-image files containing **keypoints** (pixel coordinates and scale), **descriptors** (128-d vectors), and **scores** (reliability values).
- `tools/matcher.py`: implements brute-force L2 nearest-neighbour matching with Lowe's ratio test and simple geometric constraints (vertical epipolar tolerance and disparity sign).
- `starter_code_feature.py`: performs feature extraction and matching, computes disparity and depth using stereo calibration, writes the sparse depth predictions to `P3_result.txt`, and generates visualizations.

- `tools/calculate_depth_error.py`: calculate depth errors from the ground truth depth for training images.

Feature extraction (`tools/R2D2.py`) The R2D2 extractor is invoked (via `r2d2/extract.py`) to produce a sidecar `.r2d2` file per image. Each file stores arrays loaded with NumPy: keypoints $\in \mathbb{R}^{N \times 3}$, descriptors $\in \mathbb{R}^{N \times 128}$, scores $\in \mathbb{R}^N$. Keypoints are in pixel coordinates (u, v) and include a scale value.

Descriptor matching (`tools/matcher.py`) Let L denote left descriptors and R denote right descriptors. Matching proceeds as follows:

1. Compute the pairwise L2 distance matrix $D \in \mathbb{R}^{|L| \times |R|}$ with entries $D_{ij} = \|d_i^L - d_j^R\|_2$.
2. For each left descriptor find the nearest and second-nearest right descriptors (j_1, j_2) and associated distances $d_0 = D_{i,j_1}$ and $d_1 = D_{i,j_2}$.
3. Apply Lowe's ratio test and a distance threshold:

$$\text{ratio_mask}_i = (d_0/d_1 < \tau_{\text{ratio}}) \wedge (d_0 \leq \tau_{\text{dist}}).$$

Default code parameters: $\tau_{\text{ratio}} = 0.9$, $\tau_{\text{dist}} = 5.0$.

4. Enforce an epipolar constraint: $|v_i^L - v_{j_1}^R| < \epsilon_y$ with $\epsilon_y = 2$.
5. Filter out negative disparities: $u_{j_1}^R < u_i^L$.
6. The final mask is the conjunction of the ratio, epipolar and disparity masks; matches and their descriptor distances are returned as arrays.

Disparity and depth (`starter_code_feature.py`) For each accepted match (left index i , right index j) the pixel coordinates are extracted and disparity is computed as

$$\text{disparity} = u_L - u_R.$$

Depth is then estimated from stereo geometry via

$$z = \frac{f \cdot B}{\text{disparity}},$$

where f is the focal length and B is the baseline obtained by factorizing the projection matrices (from the built-in classes: `krt_from_p` and `get_stereo_calibration`).

Filtering rules As stated in the handout, to avoid invalid or very noisy depth estimates, the implementation applies the following filters:

- Skip matches with disparity ≤ 0.1 (prevents division by zero).
- Discard depths $z > 80$ m or $z < 0$ (long-range predictions are unreliable for evaluation).

Output and visualization Filtered sparse depth predictions are written to `P3_result.txt` with the format `(image_id u v disparity depth)`. Visualizations are created using OpenCV's `cv.drawMatches`.

Evaluation of Estimated Depth Error Using Epipolar Constraint on Training Set Parameters used: `distance_ratio:0.82`, `distance_threshold:2.0`

- Mean Absolute Error (MAE): 1.3131
- Root Mean Squared Error (RMSE): 2.9181
- Relative Error: 0.0575

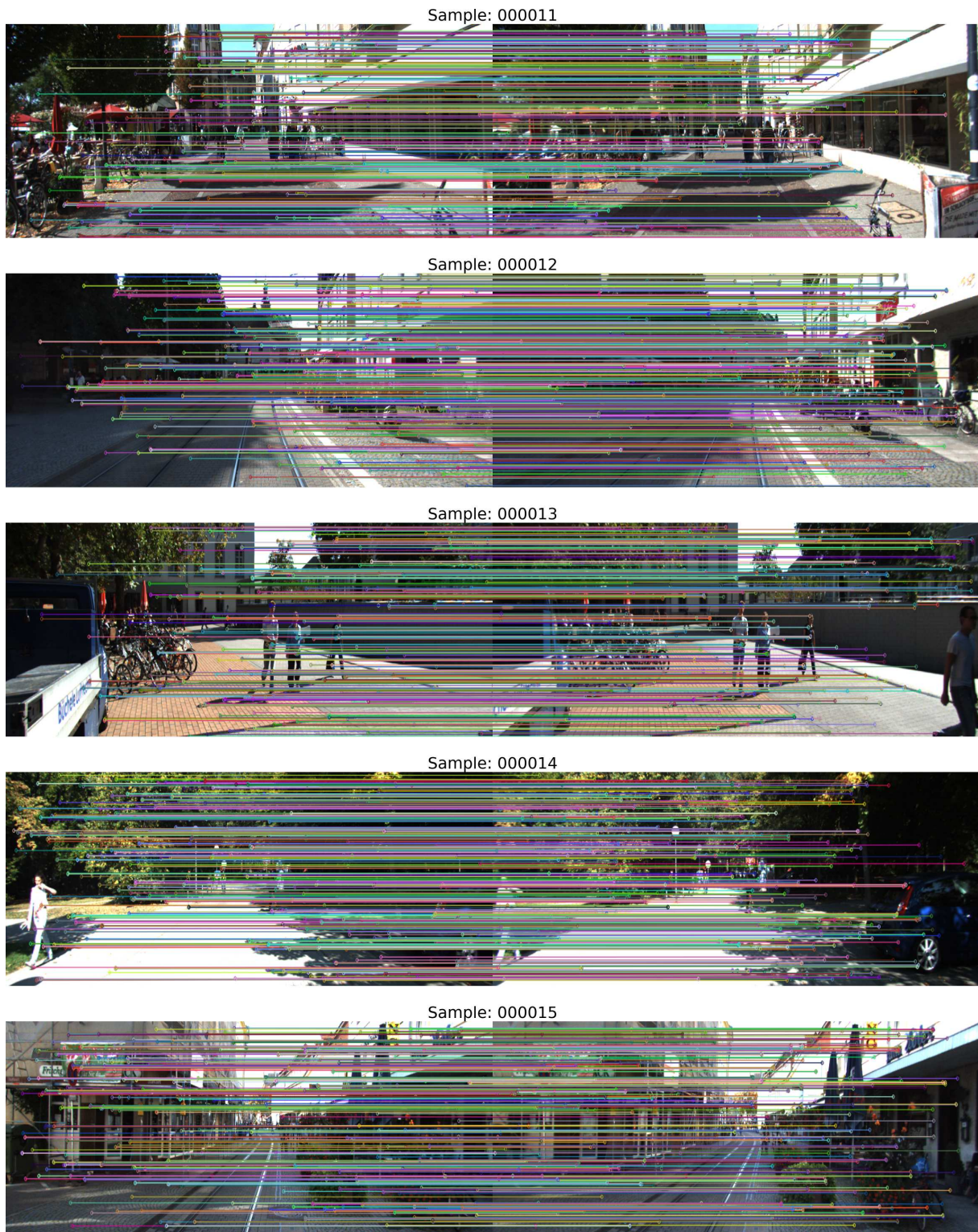


Figure 6: Feature matching on the test images using brute-force L2 nearest-neighbour matching and epipolar constraints with R2D2 [1].

1.1.3 Outlier Rejection

RANSAC-based Outlier Rejection To improve geometric consistency beyond the simple epipolar vertical-tolerance test, I ran RANSAC to estimate the fundamental matrix

from tentative descriptor matches and reject the outliers. The added RANSAC stage performs the following steps:

1. Build tentative matches using brute-force L2 nearest-neighbour matching with Lowe's ratio test. (Same as the previous approach.)
2. Convert matched pixel coordinates to homogeneous points:

$$\mathbf{x}_1 = [u_1, v_1, 1]^T, \quad \mathbf{x}_2 = [u_2, v_2, 1]^T.$$

3. Estimate the fundamental matrix F using RANSAC with the `cv.findFundamentalMat()` function:

$$F = \text{findFundamentalMat}(\{\mathbf{x}_1\}, \{\mathbf{x}_2\}; \text{method} = \text{FM_RANSAC}, \tau_{\text{ransac}}, \text{conf}).$$

4. Use the returned inlier mask from RANSAC to filter matches for downstream disparity and depth computation.

Evaluation of Estimated Depth Error Using RANSAC on Training Set Parameters used: `distance_ratio:0.82`, `distance_threshold:2.0`, `ransac_reproj_threshold:1`, `confidence:0.99`

- Mean Absolute Error (MAE): 1.2295
- Root Mean Squared Error (RMSE): 2.7331
- Relative Error: 0.0563

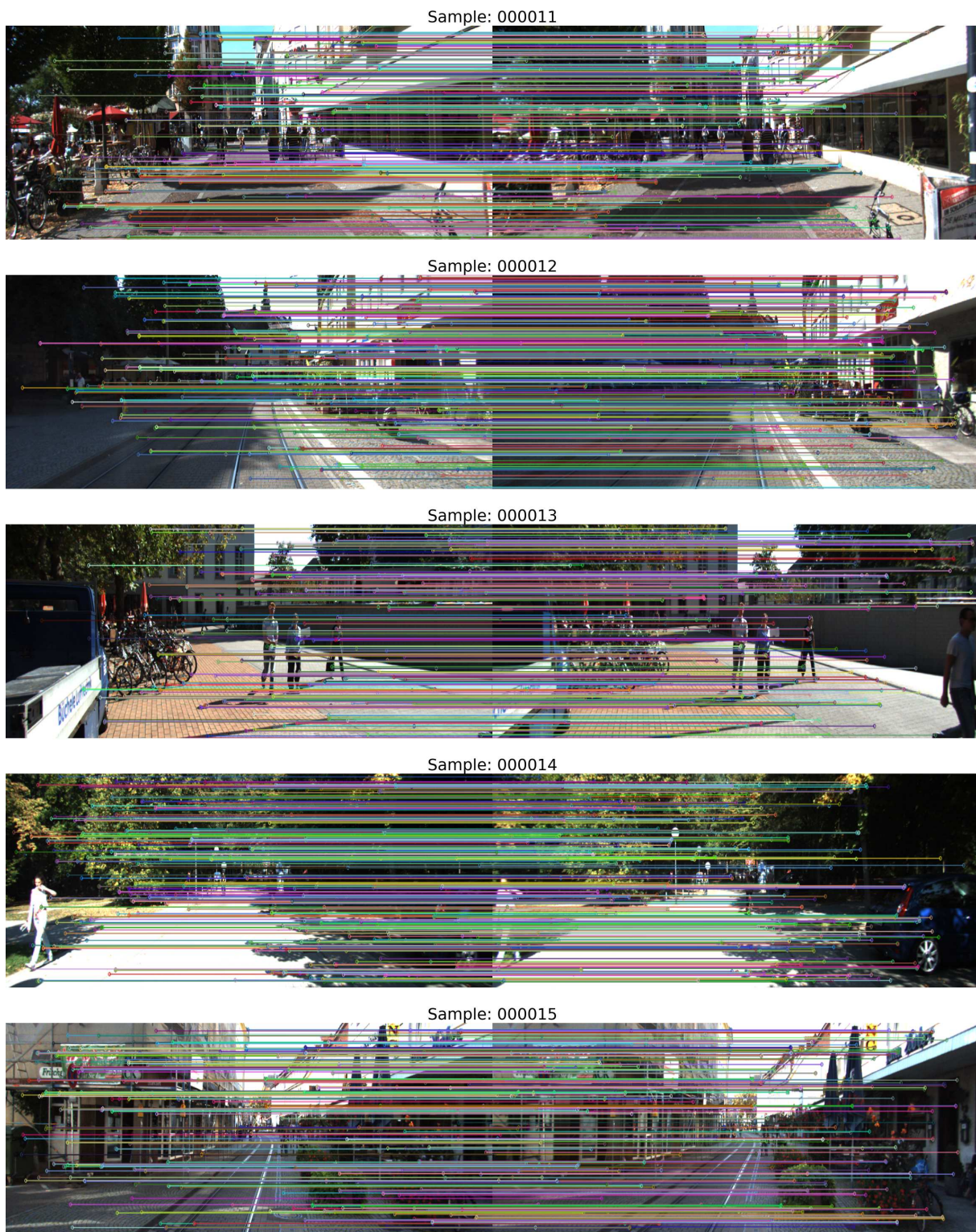


Figure 7: Feature matching on the test images using brute-force L2 nearest-neighbour matching and RANSAC outlier rejection with R2D2 [1].

1.1.4 Alternative Algorithm

For alternative algorithms for image matching, two approaches will be implemented.

ORB: Feature Detector Summary First, I implemented an alternative pipeline that replaces the learned R2D2 detector with the classical ORB detector/descriptor. ORB (Oriented FAST and Rotated BRIEF) [2] detects keypoints using the FAST corner detector applied on an image pyramid for scale robustness, and estimates a dominant orientation for each keypoint to obtain rotation invariance, and computes BRIEF-style binary descriptors on a rotated sampling pattern aligned to that orientation. The resulting 256-bit binary vectors are compact and extremely fast to match using Hamming distance; ORB also applies non-maximum suppression and keypoint scoring to retain stable points across scales, making it suitable for real-time or resource-constrained pipelines where learned descriptors may be too costly. As ORB produces binary descriptors and keypoints with fewer metadata fields than R2D2, a small amount of code changes is required to feed ORB output into the same downstream matching and depth estimation pipeline. The main changes are summarized below.

ORB: Code-level adaptations The following steps describe the minimal adapter implemented in `starter_code_feature.py`:

1. Detect and compute ORB keypoints and descriptors: `kp, desc = orb.detectAndCompute(img, None)`.
2. Convert the list of `cv2.KeyPoint` objects to a NumPy array of pixel coordinates: `pts = np.array([k.pt for k in kp], dtype=float)` (shape $N \times 2$).
3. Extract a per-keypoint score from ORB's `kp.response`.
4. A Hamming-capable matching method was introduced so binary descriptors from ORB can be matched natively without converting them to floating point. Specifically, the descriptors are kept as `uint8` and matched with OpenCV's brute-force matcher (e.g. `cv::BFMatcher(cv::NORM_HAMMING)`). I still applied Lowe's ratio test and an empirical distance threshold to remove weak matches.
5. Initialize the defined `FeatureMatcher` with the adapted arrays: `FeatureMatcher(pts, desc, scores, ...)`.

ORB: Evaluation ORB is faster but typically less robust than the learned R2D2 features in challenging illumination or viewpoint changes. Parameters used: `distance_ratio:0.9`, `distance_threshold:45.0`, `ransac_reproj_thershhold:1`, `confidence:0.99`

- Mean Absolute Error (MAE): 2.4556
- Root Mean Squared Error (RMSE): 4.6676
- Relative Error: 0.0795

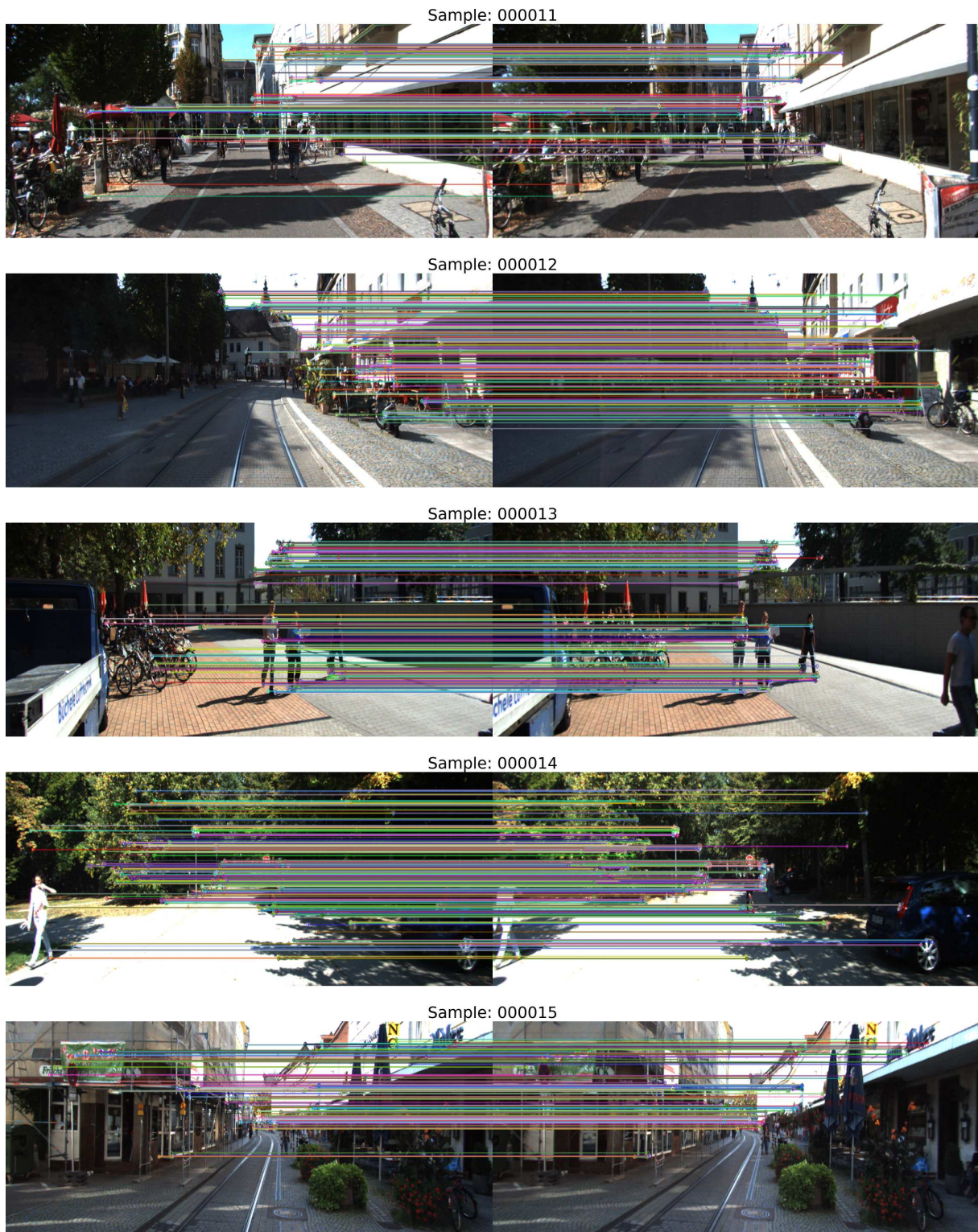


Figure 8: Feature matching on the test images using brute-force nearest-neighbour matching and RANSAC outlier rejection with ORB.

The second alternative algorithm is to use a fully learned Graph-based neural-network framework [SuperGlue](#) [3], that combines with an Optimal Matching layer that is trained to perform matching on two sets of sparse image features. In this network, it natively uses the output from [SuperPoint](#) [4], and to simplify the process, I'll use the demo script

in SuperGlue’s repository to produce keypoint pairs and descriptors.

SuperGlue: Algorithm summary SuperGlue is a learned, graph-based matching module that replaces heuristic nearest-neighbour matching with a learned attention and optimal-matching stage. In short:

- Input: two sets of sparse features $\{(x_i, d_i)\}_{i=1}^N$ and $\{(y_j, d'_j)\}_{j=1}^M$ (pixel coordinates and descriptors).
- Contextualization: descriptors are augmented with positional encodings and processed by self- and cross-attention layers to produce context-aware descriptors c_i and c'_j .
- Score matrix: a learned similarity produces scores $S_{ij} = \exp(c_i^\top c'_j)$ (optionally with a learned linear projection and temperature).
- Optimal matching: Sinkhorn normalization is applied to S to produce a (soft) doubly-stochastic matrix $P = \text{Sinkhorn}(S)$. Discrete matches are then taken as

$$m(i) = \arg \max_j P_{ij} \quad \text{with} \quad P_{i,m(i)} > \tau_{\text{match}},$$

and match confidence is given by $\max_j P_{ij}$.

- Output: one-to-one consistent matches with per-match confidence; the learned module handles ambiguity and enforces global consistency better than independent NN tests.

The approach is described in detail in the SuperGlue paper [3]. In this project we use the pretrained demo weights (“outdoor”) and the Sinkhorn-based matching with a small confidence threshold; the resulting matches are passed unchanged to the depth pipeline described above.

SuperGlue: Code-level adaptations Integration of SuperGlue into the pipeline was implemented as a separate module to avoid perturbing the existing detector/descriptor flow.

1. Batch invocation and file-format handling. I ran the provided demo script (`match_pairs.py`) once over all image pairs (via a `pairs.txt` file) so SuperGlue produces a set of `*.npz` outputs in a single output folder. Each `.npz` contains the arrays `keypoints0`, `keypoints1`, `matches` and `match_confidence`. A loader in `tools/superglue.py` reads those files and extracts only the matches where `matches[i] > -1` and `match_confidence[i]` exceeds a small confidence threshold.
2. Adapter to the sparse depth pipeline. SuperGlue returns keypoint coordinates in pixel space and indices of matched points, so I converted those directly to the pipeline’s expected format: left/right pixel coordinates (u,v), compute disparity $u_L - u_R$, apply the same disparity / depth filters (disparity > 0.1 , $0 < z \leq 80$ m), and write valid predictions to `P3_result.txt`.

SuperGlue: Evaluation

- Mean Absolute Error (MAE): 1.8
- Root Mean Squared Error (RMSE): 4.3443
- Relative Error: 0.0861



Figure 9: Feature matching on the test images using the SuperGlue framework.

1.2 Question 2: 3D Point Cloud Registration

1.2.1 Nearest Neighbour Search

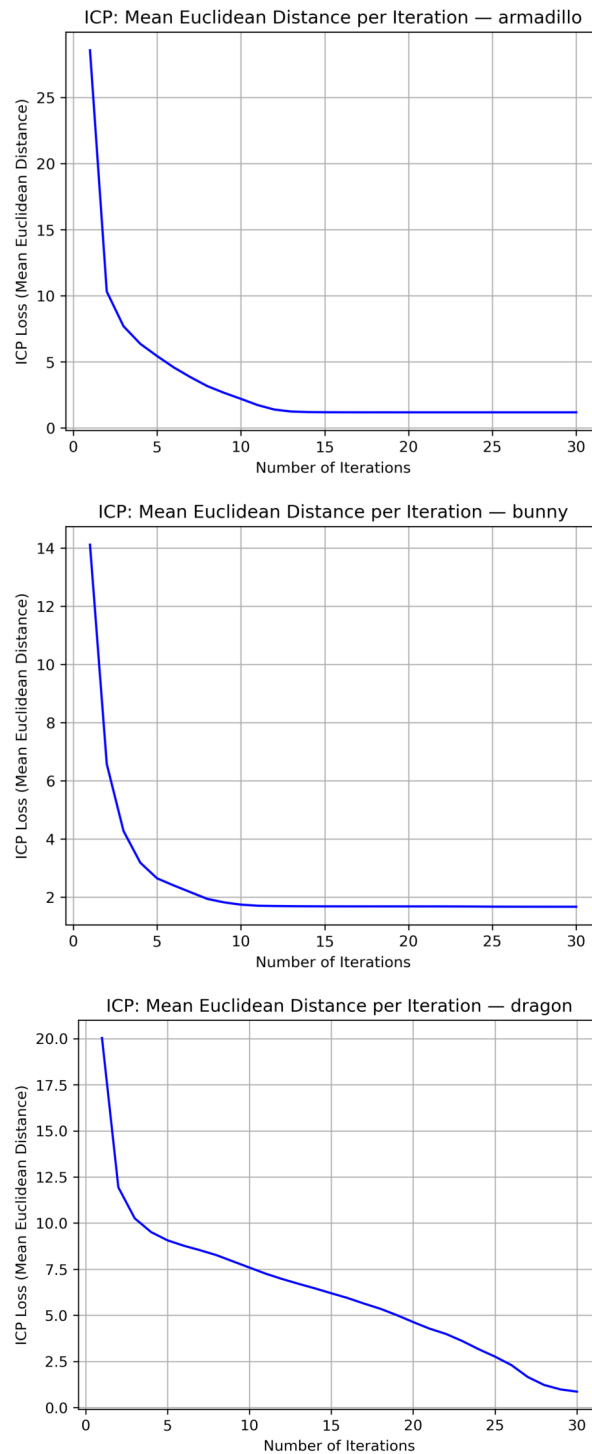


Figure 10: ICP loss for each object during iterations.

Description of the ICP loss From Fig. 10, the ICP loss (mean Euclidean distance between corresponding point pairs) for all objects decreases sharply within the first five iterations. For the Armadillo and Bunny datasets, the loss stabilizes around 1 and 2,

respectively, after approximately ten iterations. In contrast, the Dragon dataset exhibits a more gradual, nearly linear decline between the 5th and 25th iteration, reaching near zero by the 30th iteration. Additional iterations could be tested on the Dragon dataset to confirm whether it has fully converged or if further improvement is possible.

Explanation of the ICP loss The differing convergence behaviors in the ICP loss plots can be attributed primarily to variations in the point density of the target clouds, since the number of source point clouds for all objects are approximately 450, which can be excluded as a factor. The Armadillo and Bunny datasets, each containing around 1400 target points, provide dense coverage for ICP to quickly establish correspondences and converge within the first few iterations. In contrast, the Dragon dataset contains about 1200 target points, resulting in a sparser surface coverage. This reduced density limits the number of reliable correspondences, causing ICP to refine the alignment more gradually. Consequently, the dragon's loss decreases in a slower, near-linear trend.

1.2.2 Pose Estimation from the Correspondences

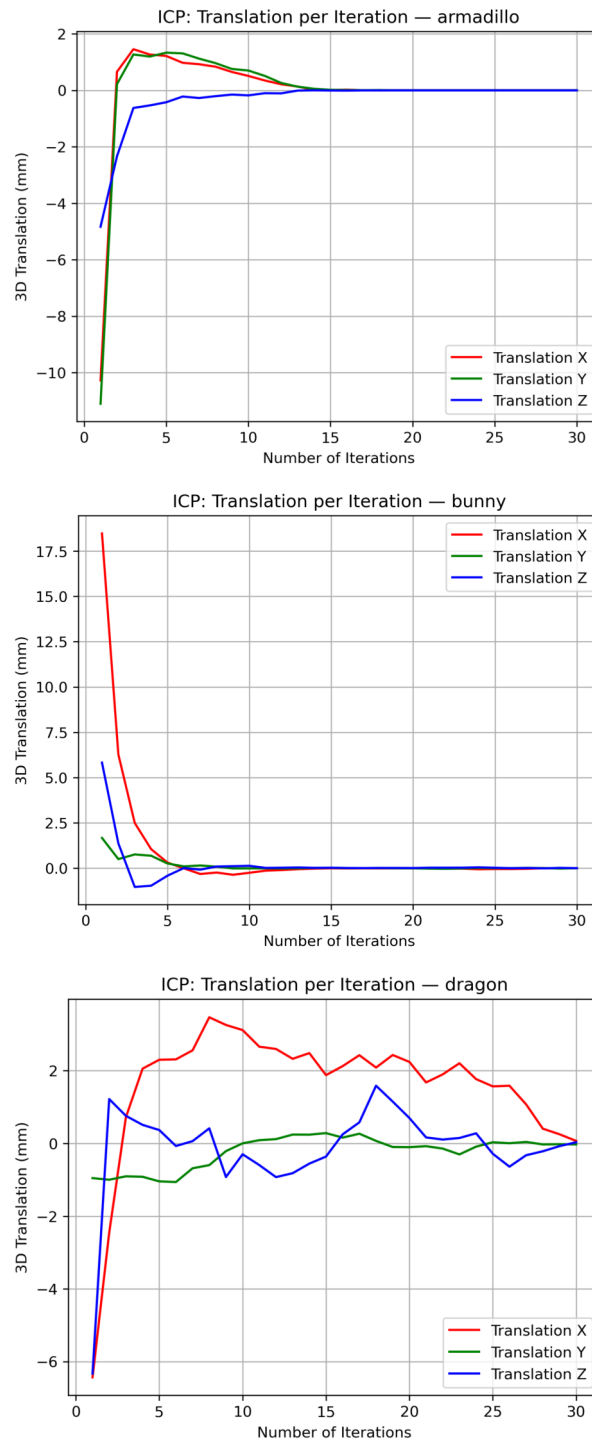


Figure 11: 3D Translation for each object during iterations.

Description of the estimated 6D pose changes during ICP iterations From Fig. 11, the evolution of the estimated 6D pose during the ICP iterations shows that most translation updates occur within the first few iterations. For the armadillo and bunny datasets, the translation components (X, Y, Z) change sharply at the beginning as the source cloud aligns with the target's global position, then quickly stabilize near zero,

indicating convergence. In contrast, the dragon dataset displays persistent oscillations, especially along the X-axis, where the translation continues to vary even after 20 iterations.

Explanation of the estimated 6D pose changes during ICP iterations This behavior reflects the convergence dynamics of ICP. The rapid initial changes correspond to large corrective updates that reduce coarse misalignment, while the later stabilization indicates consistent correspondences and refined alignment. The fluctuations in the dragon dataset arise from its lower target point density that was mentioned above, which limits the number of reliable correspondences. As a result, ICP requires more iterations and exhibits a less stable pose estimation process before convergence for the Dragon dataset.

1.2.3 Iterative Closest Point Registration

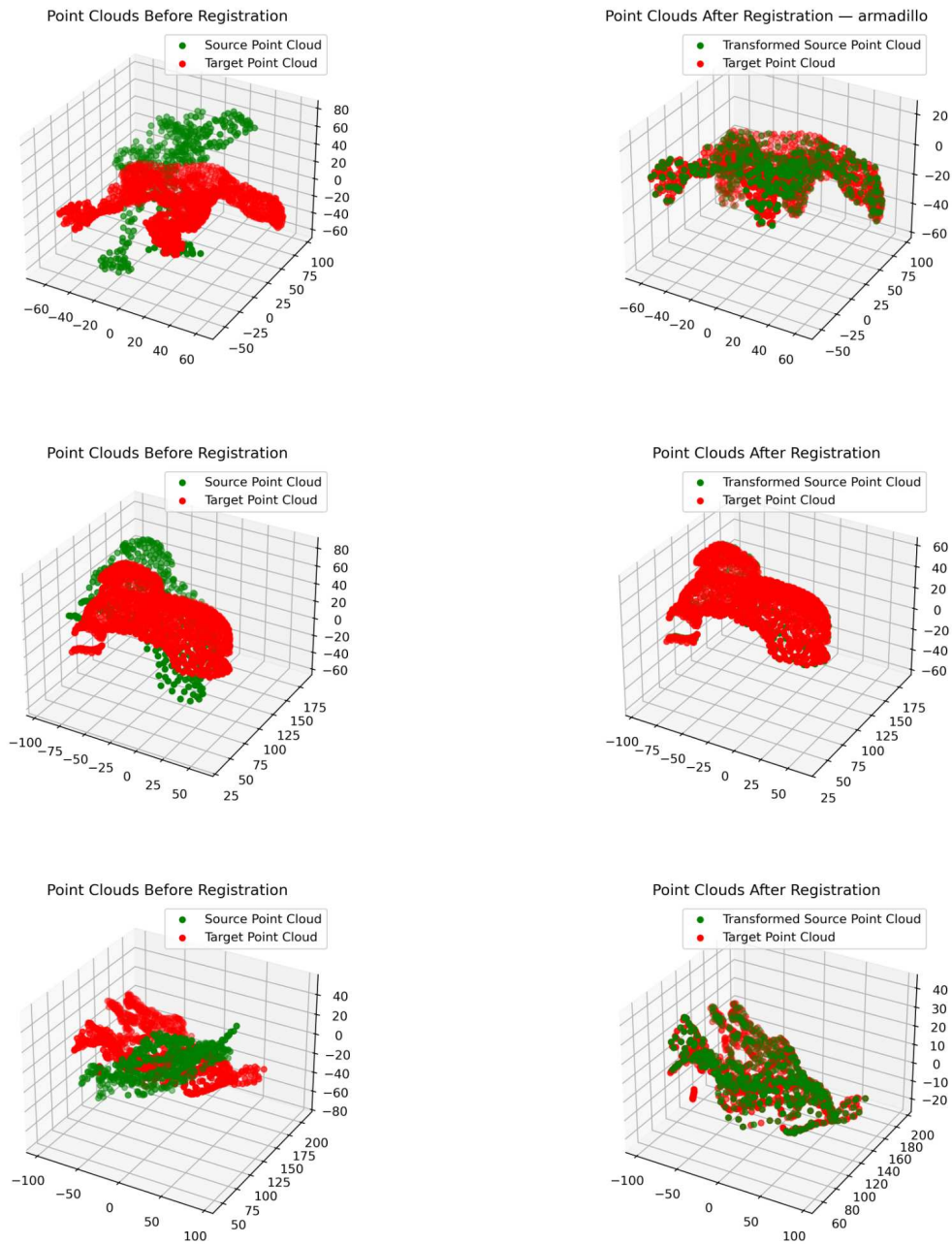


Figure 12: Demonstration of the point cloud registration result using the ICP algorithm after 30 iterations.

```
bunny -- Rotation error (deg): 0.0000, Translation error: 0.0001
dragon -- Rotation error (deg): 0.0000, Translation error: 0.0000

armadillo -- Estimated 6D pose:
[[ 0.777541 -0.066073  0.625351 -2.689722]
 [-0.384049  0.737557  0.555442 -1.858343]
 [-0.497931 -0.672045  0.548106 -5.961925]
 [ 0.          0.          0.          1.          ]]
```

Listing 1: Estimated rotation and translation errors on training sets, and estimated test set 6D pose

References

- [1] J. Revaud, P. Weinzaepfel, C. R. de Souza, and M. Humenberger, “R2D2: repeatable and reliable detector and descriptor,” in *NeurIPS*, 2019.
- [2] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “Orb: An efficient alternative to sift or surf,” in *2011 International Conference on Computer Vision*, pp. 2564–2571, 2011.
- [3] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperGlue: Learning feature matching with graph neural networks,” in *CVPR*, 2020.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “Superpoint: Self-supervised interest point detection and description,” *CoRR*, vol. abs/1712.07629, 2017.